MODULE 1

Chapter 1: Databases and Database Users

1.1 INTRODUCTION

Databases and database technology are having a major impact on the growing use of computers. It is fair to say that databases play a critical role in almost all areas where computers are used, including business, engineering, medicine, law, education, and library science, to name a few. The word *database* is in such common use that we must begin by defining a database. Our initial definition is quite general.

A **database** is a collection of related data . By **data,** we mean known facts that can be recorded and that have implicit meaning. For example, consider the names, telephone numbers, and addresses of the people you know. You may have recorded this data in an indexed address book, or you may have stored it on a diskette, using a personal computer and software such as DBASE IV or V, Microsoft ACCESS, or EXCEL. This is a collection of related data with an implicit meaning and hence is a database.

A database has the following implicit properties:

• A database represents some aspect of the real world, sometimes called the **miniworld** or the **Universe of Discourse (UoD).** Changes to the miniworld are reflected in the database.

• A database is a logically coherent collection of data with some inherent meaning. A random assortment of data cannot correctly be referred to as a database.

• A database is designed, built, and populated with data for a specific purpose. It has an intended group of users and some preconceived applications in which these users are interested.

A database can be of any size and of varying complexity. For example, the list of names and addresses referred to earlier may consist of only a few hundred records, each with a simple structure. On the other hand, the card catalog of a large library may contain half a million cards stored under different categories. A database may be generated and maintained manually or it may be computerized.

A **database management system (DBMS)** is a collection of programs that enables users to create and maintain a database. The DBMS is hence a *general-purpose software system* that facilitates the processes of *defining*, *constructing*, and *manipulating* databases for various applications. **Defining** a database involves specifying the data types, structures, and constraints for the data to be stored in the database. **Constructing** the database is the process of storing the data itself on some storage medium that is controlled by the DBMS. **Manipulating** a database includes such functions as querying the database to retrieve specific data, updating the database to reflect changes in the miniworld, and generating reports from the data.

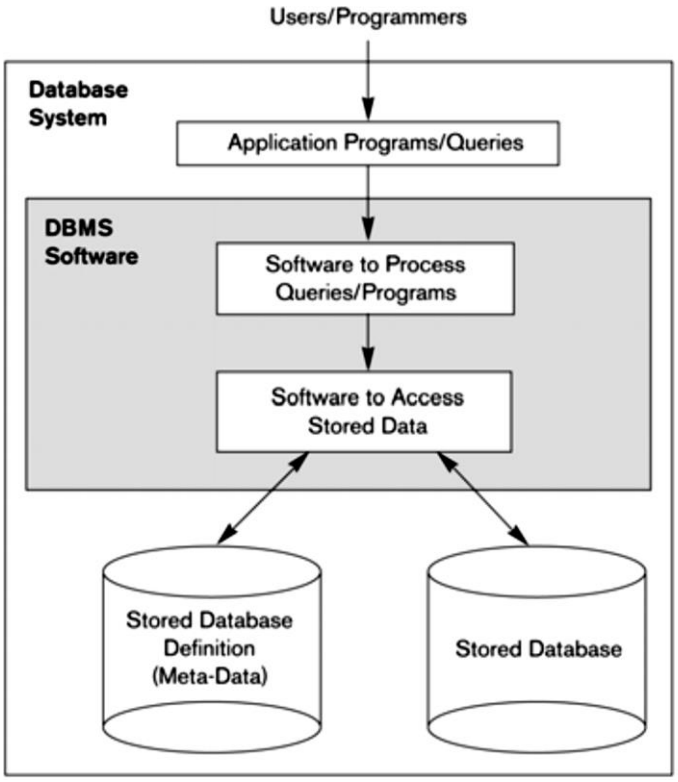


Figure 1.1
A simplified database system environment.

Example

Let us consider an example that most readers may be familiar with: a UNIVERSITY database for maintaining information concerning students, courses, and grades in a university environment. Figure shows the database structure and a few sample data for such a database. The database is organized as five files, each of which stores data records of the same type (Note 2). The STUDENT file stores data on each student; the COURSE file stores data on each course; the SECTION file stores data on each section of a course; the GRADE_REPORT file stores the grades that students receive in the various sections they have completed; and the PREREQUISITE file stores the prerequisites of each course.

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	04	King
92	CS1310	Fall	04	Anderson
102	CS3320	Spring	05	Knuth
112	MATH2410	Fall	05	Chang
119	CS1310	Fall	05	Anderson
135	CS3380	Fall	05	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Figure 1.2
A database that stores student and course information.

1.3 CHARACTERISTICS OF THE DATABASE APPROACH

1.3.1 Self-Describing Nature of a Database System
1.3.2 Insulation between Programs and Data, and Data Abstraction
1.3.3 Support of Multiple Views of the Data
1.3.4 Sharing of Data and Multiuser Transaction Processing

A number of characteristics distinguish the database approach from the traditional approach of programming with files. In traditional **file processing,** each user defines and implements the files needed for a specific application as part of programming the application. For example, one user, the *grade reporting office,* may keep a file on students and their grades. Programs to print a student's transcript and to enter new grades into the file are implemented. A second user, the accounting office, may keep track of students' fees and their payments. Although both users are interested in data about students, each user maintains separate files—and programs to manipulate these files—because each requires some data not available from the other user's files. This redundancy in defining and storing data results in wasted storage space and in redundant efforts to maintain common data up-to-date.
In **the database approach**, a **single repository** of data is maintained that is defined once and then is accessed by various users.

The main characteristics of the database approach versus the file-processing approach are the following

1.3.1 Self-Describing Nature of a Database System

A fundamental characteristic of the database approach is that the database system contains not only the database itself but also a complete definition or description of the database structure and constraints. This definition is stored in the system **catalog,** which contains information such as the structure of each file, the type and storage format of each data item, and various constraints on the data. The information stored in the catalog is called **meta-data,** and it describes the structure of the primary database.

In traditional file processing, data definition is typically part of the application programs themselves. Hence, these programs are constrained to work with only *one specific database,* whose structure is declared in the application programs. For example, a PASCAL program may have record structures declared in it; a C++ program may have "struct" or "class" declarations

1.3.2 Insulation between Programs and Data, and Data Abstraction

In traditional file processing, the structure of data files is embedded in the access programs, so any changes to the structure of a file may require changing all programs that access this file. By contrast, DBMS access programs do not require such changes in most cases. The structure of data files is stored in the DBMS catalog separately from the access programs. We call this property **program-data independence.**

In object-oriented and object-relational databases users can define operations on data as part of the database definitions. An **operation** (also called a function) is specified in two parts. The **interface** (or signature) of an operation includes the operation name and the data types of its arguments (or parameters). The **implementation** (or method) of the operation is specified separately and can be changed without affecting the interface. User application programs can operate on the data by invoking these operations through their names and arguments, regardless of how the operations are implemented. This may be termed **program-operation independence.**

The characteristic that allows program-data independence and program-operation independence is called **data abstraction.** A data model is a type of data abstraction that is used to provide this conceptual representation.

1.3.3 Support of Multiple Views of the Data

A database typically has many users, each of whom may require a **different perspective or view of the database.** A view may be a subset of the database or it may contain virtual data that is derived from the database files but is not explicitly stored. Some users may not need to be aware of whether the data they refer to is stored or derived.

1.3.4 Sharing of Data and Multiuser Transaction Processing

A multiuser DBMS, as its name implies, must allow **multiple users to access the database at the same time**. This is essential if data for multiple applications is to be integrated and maintained in a single database. The DBMS must include **concurrency control software** to ensure that several users trying to update the same data do so in a controlled manner so that the result of the updates is correct. For example, when several reservation clerks try to assign a seat on an airline flight, the DBMS should ensure that each seat can be accessed by only one clerk at a time for assignment to a passenger. These types of applications are generally called **on-line transaction processing (OLTP) application**s. A fundamental role of multiuser DBMS software is to ensure that concurrent transactions operate correctly.

1.4 ADVANTAGE OF USING A DBMS

1 Controlling Redundancy
2 Restricting Unauthorized Access
3 Providing Persistent Storage for Program Objects and Data Structures
4 Permitting Inferencing and Actions Using Rules
5 Providing Multiple User Interfaces
6 Representing Complex Relationships Among Data
7 Enforcing Integrity Constraints
8 Providing Backup and Recovery
9 Additional Implications of the Database Approach

1. Controlling Redundancy

In traditional software development utilizing file processing, every user group maintains its own files for handling its data-processing applications. For example, consider the UNIVERSITY database example two groups of users might be the course registration personnel and the accounting office. In the traditional approach, each group independently keeps files on students. The accounting office also keeps data on registration and related billing information, whereas the registration office keeps track of student courses and grades. Much of the data is stored twice: once in the files of each user group. Additional user groups may further duplicate some or all of the same data in their own files.

This **redundancy** in storing the same data multiple times leads to several problems. **First**, there is the need to perform a single logical update—such as entering data on a new student—multiple times: once for each file where student data is recorded. This leads to *duplication of effort.* **Second,** *storage space is wasted* when the same data is stored repeatedly, and this problem may be serious for large databases. **Third,** files that represent the same data may become *inconsistent.* This may happen because an update is applied to some of the files but not to others.

In the database approach, the views of different user groups are integrated during database design. For consistency, we should have a database design that stores each logical data item—such as a student's name or birth date—in *only one place* in the database. This does not permit inconsistency, and it saves storage space.

2. Restricting Unauthorized Access

When multiple users share a database, it is likely that some users will not be authorized to access all information in the database. For example, financial data is often considered confidential, and hence only authorized persons are allowed to access such data. In addition, some users may be permitted only to retrieve data, whereas others are allowed both to retrieve and to update. Hence, the type of access operation—retrieval or update—must also be controlled. Typically, users or user groups are given account numbers protected by passwords, which they can use to gain access to the database.

3. Providing Persistent Storage for Program Objects and Data Structures

Databases can be used to provide **persistent storage** for program objects and data structures. This is one of the main reasons for the emergence of the **object-oriented database systems.**

Programming languages typically have complex data structures, such as record types in PASCAL or class definitions in C++. The values of program variables are discarded once a program terminates.

The persistent storage of program objects and data structures is an important function of database systems. Traditional database systems often suffered from the so-called **impedance mismatch problem,** since the data structures provided by the DBMS were incompatible with the programming language's data structures. Object-oriented database systems typically offer data structure **compatibility** with one or more object-oriented programming languages.

4. Permitting Inferencing and Actions Using Rules

Some database systems provide capabilities for defining *deduction rules* for *inferencing* new information from the stored database facts. Such systems are called **deductive database systems.** For example, there may be complex rules in the miniworld application for determining when a student is on probation. These can be specified *declaratively* as **rules,** which when compiled and maintained by the DBMS can determine all students on probation.

5. Providing Multiple User Interfaces

Because many types of users with varying levels of technical knowledge use a database, a DBMS should provide a variety of user interfaces. These include query languages for casual users; programming language interfaces for application programmers; forms and command codes for parametric users; and menu-driven interfaces and natural language interfaces for stand-alone users. Both forms-style interfaces and menu-driven interfaces are commonly known as **graphical user interfaces (GUIs).**

6 Representing Complex Relationships Among Data

A database may include numerous varieties of data that are interrelated in many ways. A DBMS must have the capability to represent a variety of complex relationships among the data as well as to retrieve and update related data easily and efficiently.

7 Enforcing Integrity Constraints

Most database applications have certain **integrity constraints** that must hold for the data. A DBMS should provide capabilities for defining and enforcing these constraints. The simplest type of integrity constraint involves specifying a data type for each data item.

For example we may specify that the value of the Class data item within each student record must be an integer between 1 and 5 and that the value of Name must be a string of no more than 30 alphabetic characters.

8 Providing Backup and Recovery

A DBMS must provide facilities for recovering from hardware or software failures. The **backup and recovery subsystem** of the DBMS is responsible for recovery. For example, if the computer system fails in the middle of a complex update program, the recovery subsystem is responsible for making sure that the database is restored to the state it was in before the program started executing.

9 Additional Implications of the Database Approach

- Potential for Enforcing Standards
- Reduced Application Development Time
- Flexibility
- Availability of Up-to-Date Information
- Economies of Scale

Potential for Enforcing Standards

The Database approach permits the DBA to define and enforce standards among database users in a large organization. Standards can be defined for names and formats of data elements, display formats, report structures, terminology, and so on.

Reduced Application Development Time

A prime selling feature of the database approach is that developing a new application—such as the retrieval of certain data from the database for printing a new report—takes very little time. Designing and implementing a new database from scratch may take more time than writing a single specialized file application

Flexibility

It may be necessary to change the structure of a database as requirements change. Modern DBMSs allow certain types of changes to the structure of the database without affecting the stored data and the existing application programs.

Availability of Up-to-Date Information

A DBMS makes the database available to all users. As soon as one user's update is applied to the database, all other users can immediately see this update. This availability of up-to-date information is essential for many transaction-processing applications, such as reservation systems or banking databases.

Economies of Scale

The DBMS approach permits consolidation of data and applications, thus reducing the amount of wasteful overlap between activities of data-processing personnel in different projects or departments.

Chapter 2: Database System Concepts and Architecture

A **data model**—a collection of concepts that can be used to describe the structure of a database—provides the necessary means to achieve this abstraction .By *structure of a database* we mean the data types, relationships, and constraints that should hold on the data. Most data models also include a set of **basic operations** for specifying retrievals and updates on the database.
This allows the database designer to specify a set of valid **user-defined operations** that are allowed on the database objects .An example of a user-defined operation could be COMPUTE_GPA, which can be applied to a STUDENT object.

2.1.1 Categories of Data Models

A) High-level or **conceptual data models** provide concepts that are close to the way many users perceive data.
B) Low-level or **physical data models** provide concepts that describe the details of how data is stored in the computer.
C) **Representational** (or **implementation**) **data models,** which provide concepts that may be understood by end users but that are not too far removed from the way data is organized within the computer. Representational data models represent data by using record structures and hence are sometimes called **record-based data models.**

An **entity** represents a real-world object or concept, such as an employee or a project, that is described in the database. An **attribute** represents some property of interest that further describes an entity, such as the employee's name or salary. A **relationship** among two or more entities represents an interaction among the entities; for example, a works-on relationship between an employee and a project.

Representational or implementation data models are the models used most frequently in traditional commercial DBMSs, and they include the widely used **relational data model.** Legacy data models—the **network** and **hierarchical models. Object data models** are new family of higher-level implementation data models that are closer to conceptual data models.

2.1.2 Schemas, Instances, and Database State

In any data model it is important to distinguish between the ***description* of the database** and the ***database itself.*** The description of a database is called the **database schema,** which is specified during database design and is not expected to change frequently.

A displayed schema is called a **schema diagram.** A schema diagram displays only *some aspects* of a schema, such as the names of record types and data items, and some types of constraints.

STUDENT

Name	Student_number	Class	Major

COURSE

Course_name	Course_number	Credit_hours	Department

PREREQUISITE

Course_number	Prerequisite_number

SECTION

Section_identifier	Course_number	Semester	Year	Instructor

GRADE_REPORT

Student_number	Section_identifier	Grade

Figure 2.1
Schema diagram for the database in Figure 1.2.

The data in the database at a particular moment in time is called a **database state** or **snapshot.** It is also called the *current* set of **occurrences** or **instances** in the database.

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	04	King
92	CS1310	Fall	04	Anderson
102	CS3320	Spring	05	Knuth
112	MATH2410	Fall	05	Chang
119	CS1310	Fall	05	Anderson
135	CS3380	Fall	05	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Figure 1.2
A database that stores student and course information.

The DBMS is partly responsible for ensuring that *every* state of the database is a **valid state**—that is, a state that satisfies the structure and constraints specified in the schema. Hence, specifying a correct schema to the DBMS is extremely important, and the schema must be

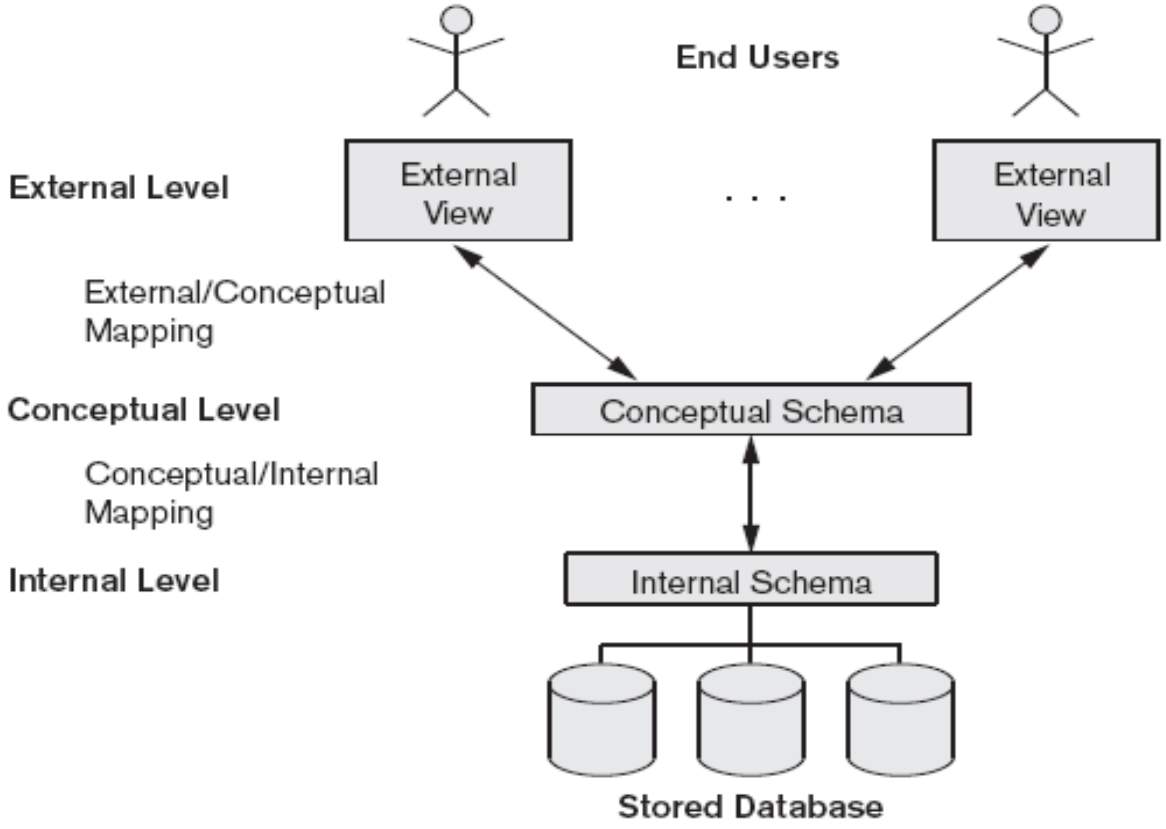
designed with the utmost care. The DBMS stores the descriptions of the schema constructs and constraints—also called the **meta-data**—in the DBMS catalog so that DBMS software can refer to the schema whenever it needs to. The schema is sometimes called the **intension,** and a database state an **extension** of the schema.

2.2 DBMS Architecture and Data Independence

In this section we specify an architecture for database systems, called the **three-schema architecture** which was proposed to help achieve and visualize the DBMS characteristics.

2.2.1 The Three-Schema Architecture

Proposed to support DBMS characteristics: Program-**data independence,** .Support of **multiple views** of the data. Not explicitly used in commercial DBMS products, but has been useful in explaining database system organization.



1. The **internal level** has an **internal schema,** which describes the physical storage structure of the database. The internal schema uses a physical data model and describes the complete details of data storage and access paths for the database.
2. The **conceptual level** has a **conceptual schema,** which describes the structure of the whole database for a community of users. The conceptual schema hides the details of physical storage structures and concentrates on describing entities, data types, relationships, user operations, and constraints. A high-level data model or an implementation data model can be used at this level.
3. The **external** or **view level** includes a number of **external schemas** or **user views.** Each external schema describes the part of the database that a particular user group is interested in

and hides the rest of the database from that user group. A high-level data model or an implementation data model can be used at this level.

DBMS must transform a request specified on an external schema into a request against the conceptual schema, and then into a request on the internal schema for processing over the stored database. If the request is a database retrieval, the data extracted from the stored database must be reformatted to match the user's external view. The processes of transforming requests and results between levels are called **mappings.** These mappings may be time-consuming, so some DBMSs—especially those that are meant to support small databases—do not support external views. Even in such systems, however, a certain amount of mapping is necessary to transform requests between the conceptual and internal levels.

2.2.2 Data Independence

The three-schema architecture can be used to explain the concept of **data independence,** which can be defined as the capacity to change the schema at one level of a database system without having to change the schema at the next higher level. We can define two types of data independence:

1. **Logical data independence** is the capacity to change the conceptual schema without having to change external schemas or application programs. We may change the conceptual schema to expand the database (by adding a record type or data item), or to reduce the database (by removing a record type or data item).
2. **Physical data independence** is the capacity to change the internal schema without having to change the conceptual (or external) schemas. Changes to the internal schema may be needed because some physical files had to be reorganized—for example, by creating additional access structures—to improve the performance of retrieval or update. If the same data as before remains in the database, we should not have to change the conceptual schema .

2.3 Database Languages and Interfaces

2.3.1 DBMS Languages

Data definition language (DDL)is used by the DBA and by database designers to define both schemas. The DBMS will have a DDL compiler whose function is to process DDL statements in order to identify descriptions of the schema constructs and to store the schema description in the DBMS catalog.the DDL is used to specify the conceptual schema only.

Storage definition language (SDL)is used to specify the internal schema. The mappings between the two schemas may be specified in either one of these languages.

View definition language (VDL) is used to specify user views and their mappings to the conceptual schema, but in most DBMSs the DDL is used to define both conceptual and external schemas.

Data manipulation language (DML) Once the database schemas are compiled and the database is populated with data, users must have some means to manipulate the database. Typical manipulations include retrieval, insertion, deletion, and modification of the data. The DBMS provides a **data manipulation language (DML)** for these purposes. There are two

main types of DMLs. A **high-level** or **nonprocedural DML** can be used on its own to specify complex database operations in a concise manner. A **low-level** or **procedural DML** *must* be embedded in a general-purpose programming language. This type of DML typically retrieves individual records or objects from the database and processes each separately. Low-level DMLs are also called **record-at-a-time DMLs.** High-level DMLs, such as SQL, can specify and retrieve many records in a single DML statement and are hence called **set-at-a-time** or **set-oriented DMLs.**

2.3.2 DBMS Interfaces

1. **Menu-Based Interfaces for Browsing**

These interfaces present the user with lists of options, called **menus,** that lead the user through the formulation of a request. Menus do away with the need to memorize the specific commands and syntax of a query language; rather, the query is composed step by step by picking options from a menu that is displayed by the system. They are often used in **browsing interfaces,** which allow a user to look through the contents of a database in an exploratory and unstructured manner.

2. **Forms-Based Interfaces**

A forms-based interface displays a **form** to each user. Users can fill out all of the form entries to insert new data, or they fill out only certain entries, in which case the DBMS will retrieve matching data for the remaining entries. Forms are usually designed and programmed for naive users as interfaces to canned transactions.

3. **Graphical User Interfaces**

A graphical interface (GUI) typically displays a schema to the user in diagrammatic form. The user can then specify a query by manipulating the diagram. In many cases, GUIs utilize both menus and forms. Most GUIs use a **pointing device,** such as a mouse, to pick certain parts of the displayed schema diagram.

4. **Natural Language Interfaces**

These interfaces accept requests written in English or some other language and attempt to "understand" them. A natural language interface usually has its own "schema," which is similar to the database conceptual schema. The natural language interface refers to the words in its schema, as well as to a set of standard words, to interpret the request.

5. **Interfaces for Parametric Users**

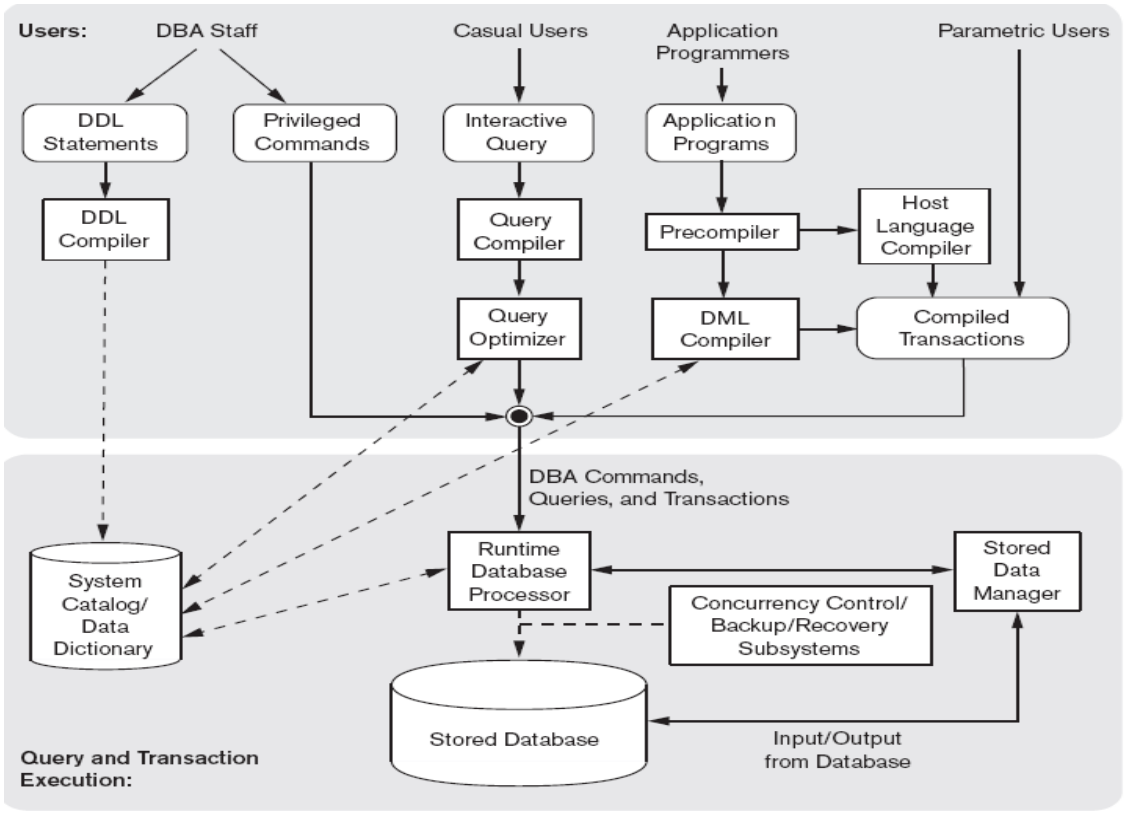
Parametric users, such as bank tellers, often have a small set of operations that they must perform repeatedly. Systems analysts and programmers design and implement a special interface for a known class of naive users. Usually, a small set of abbreviated commands is included, with the goal of minimizing the number of keystrokes required for each request

6. **Interfaces for the DBA**

Most database systems contain privileged commands that can be used only by the DBA’s staff. These include commands for creating accounts, setting system parameters, granting account authorization, changing a schema, and reorganizing the storage structures of a database.

2.4 The Database System Environment

2.4.1 DBMS Component Modules



- The top part of the figure refers to the various users of the database environment and their interfaces.
- The lower part shows the internals of the DBMS responsible for storage of data and processing of transactions.

Top part of the figure consists of **interfaces-**for the DBA staff, casual users , application programmers and parametric users.

DBA Staff works on defining the database and tuning it by making changes to its definition using the DDL and other privileged commands. The **DDL compiler** processes schema definitions, specified in the DDL, and stores descriptions of the schemas (meta-data) in the DBMS catalog. The catalog includes information such as the names of files, data items, storage details of each file and so on.

Casual Users and People with occasional need for information from database interact using the **interactive query**. Then **Query compiler** validates for correctness of the query syntax, the names of files and data elements & compiles them into an internal form and query is sent to Query optimizer. **Query optimizer** is responsible for rearrangement and possible reordering of operations, elimination of redundancies, and use of correct algorithms and indexes during execution. Makes calls on the runtime processor

Application programmers write programs in host languages such as Java, C, C++ are submitted to precompiler. **Precompiler -** extracts DML commands from an application program and sends to the DML compiler for compilation into object code for database access. Rest of the program is sent to **host language compiler.** The object codes for the DML commands and the rest of the program are linked, forming a **canned transaction** whose executable code includes calls to the runtime database processor.

Lower part of the figure consists of :

Runtime database processor executes the following with runtime parameters :

- privileged commands
- executable query plans
- canned transactions

It works with the **system catalog and stored data manager** and also with management of buffers in the main memory

The **run-time database processor** handles database accesses at run time; it receives retrieval or update **query compiler** handles high-level queries that are entered interactively. It parses, analyzes, and compiles or interprets a query by creating database access code, and then generates calls to the run-time processor for executing the code.

Stored data manager

The database and the DBMS catalog are usually stored on disk. Access to the disk is controlled primarily by the **operating system (OS),** which schedules disk input/output. A higher-level **stored data manager** module of the DBMS controls access to DBMS information that is stored on disk, whether it is part of the database or the catalog.The stored data manager may use basic OS services for carrying out low-level data transfer between the disk and computer main storage, but it controls other aspects of data transfer, such as handling buffers in main memory.

concurrency control and **backup and recovery systems** integrated into the working of the runtime database processor for purposes of transaction management.

2.4.2 Database System Utilities

Most DBMSs have **database utilities** that help the DBA in managing the database system. Common utilities have the following types of functions:

1.Loading*:* A loading utility is used to load existing data files—such as text files or sequential files—into the database. Usually, the current (source) format of the data file and the desired (target) database file structure are specified to the utility.

2. ***Backup****:* A backup utility creates a backup copy of the database, usually by dumping the entire database onto tape. The backup copy can be used to restore the database in case of

catastrophic failure. Incremental backups are also often used, where only changes since the previous backup are recorded. Incremental backup is more complex but it saves space.

3. ***File reorganization:*** This utility can be used to reorganize a database file into a different file organization to improve performance.

4. ***Performance monitoring:*** Such a utility monitors database usage and provides statistics to the DBA. The DBA uses the statistics in making decisions such as whether or not to reorganize files to improve performance .

2.4.3 Tools, Application Environments, and Communications Facilities

Other tools are often available to database designers, users, and DBAs. **CASE tools a**re used in the design phase of database systems. Another tool that can be quite useful in large organizations is an expanded **data dictionary** (or **data repository**) **system.** In addition to storing catalog information about schemas and constraints, the data dictionary stores other information, such as design decisions, usage standards, application program descriptions, and user information. Such a system is also called an **information repository.**

Application development environments, such as the PowerBuilder system, are becoming quite popular.

The DBMS also needs to interface with **communications software,** whose function is to allow users at locations remote from the database system site to access the database through computer terminals, workstations, or their local personal computers. These are connected to the database site through data communications hardware such as phone lines, long-haul networks, local-area networks, or satellite communication devices.

Chapter 3: Data Modeling Using the Entity-Relationship Model

Conceptual modeling is an important phase in designing a successful database application. Generally, the term database application refers to a particular database—for example, a BANK database that keeps track of customer accounts—and the associated programs that implement the database queries and updates—for example, programs that implement database updates corresponding to customers making deposits and withdrawals. These programs often provide user-friendly graphical user interfaces (GUIs) utilizing forms and menus. Hence, part of the database application will require the design, implementation, and testing of these application programs.

Entity-Relationship (ER) model, which is a popular high-level conceptual data model. This model and its variations are frequently used for the conceptual design of database applications, and many database design tools employ its concepts. We describe the basic data-structuring concepts and constraints of the ER model and discuss their use in the design of conceptual schemas for database applications.

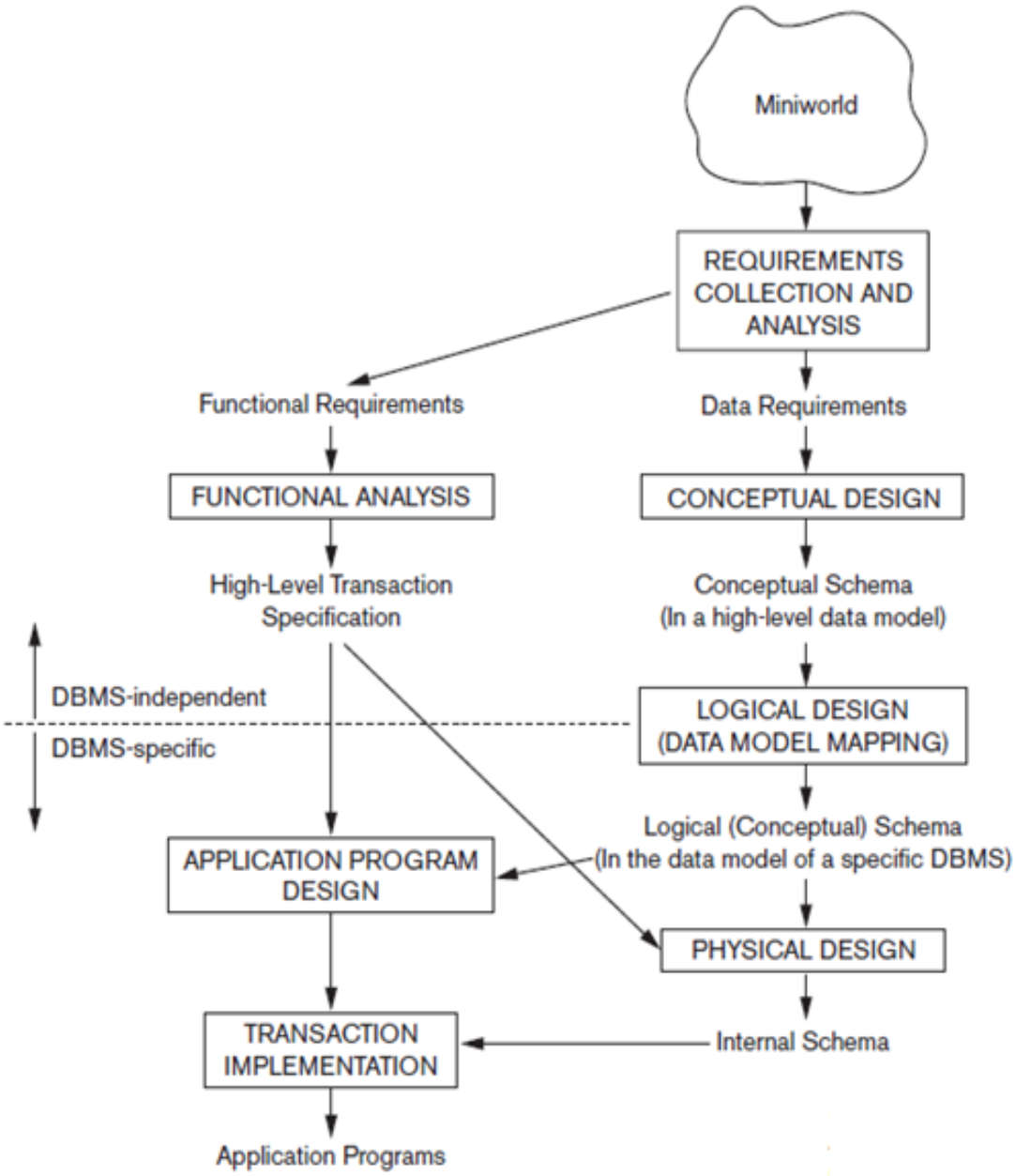
1.USING HIGH-LEVEL CONCEPTUAL DATA MODELS FOR DATABASE DESIGN

Figure shows a simplified description of the database design process. The **first step** shown is **requirements collection and analysis.** During this step, the database designers interview prospective database users to understand and document their **data requirements.** The result of this step is a concisely written set of users' requirements. These requirements should be specified in as detailed and complete a form as possible. In parallel with specifying the data requirements, it is useful to specify the known **functional requirements** of the application. These consist of the user-defined **operations** (or **transactions**) that will be applied to the database, and they include both retrievals and updates.

Once all the requirements have been collected and analysed, the next step is to create a **conceptual schema** for the database, using a high-level conceptual data model. This step is called **conceptual design.** The conceptual schema is a concise description of the data requirements of the users and includes detailed descriptions of the entity types, relationships, and constraints; these are expressed using the concepts provided by the high-level data model. Because these concepts do not include implementation details, they are usually easier to understand and can be used to communicate with nontechnical users.

During or after the conceptual schema design, the basic data model operations can be used to specify the high-level user operations identified during functional analysis. This also serves to confirm that the conceptual schema meets all the identified functional requirements. Modifications to the conceptual schema can be introduced if some functional requirements cannot be specified in the initial schema.

The next step in database design is the actual implementation of the database, using a commercial DBMS. Most current commercial DBMSs use an implementation data model—such as the relational or the object database model—so the conceptual schema is transformed from the high-level data model into the implementation data model. This step is called **logical design** or **data model mapping,** and its result is a database schema in the implementation data model of the DBMS.



Finally, the last step is the **physical design** phase, during which the internal storage structures, access paths, and file organizations for the database files are specified. In parallel with these activities, application programs are designed and implemented as database transactions corresponding to the high-level transaction specifications

2 AN EXAMPLE DATABASE APPLICATION

In this section we describe an example database application, called COMPANY, which serves to illustrate the ER model concepts and their use in schema design.

1. The company is organized into departments. Each department has a unique name, a unique number, and a particular employee who manages the department. We keep track of the start

date when that employee began managing the department. A department may have several locations.

2. A department controls a number of projects, each of which has a unique name, a unique number, and a single location.

3. We store each employee's name, social security number (Note 1), address, salary, sex, and birth date. An employee is assigned to one department but may work on several projects, which are not necessarily controlled by the same department. We keep track of the number of hours per week that an employee works on each project. We also keep track of the direct supervisor of each employee.

4. We want to keep track of the dependents of each employee for insurance purposes. We keep each dependent's first name, sex, birth date, and relationship to the employee.

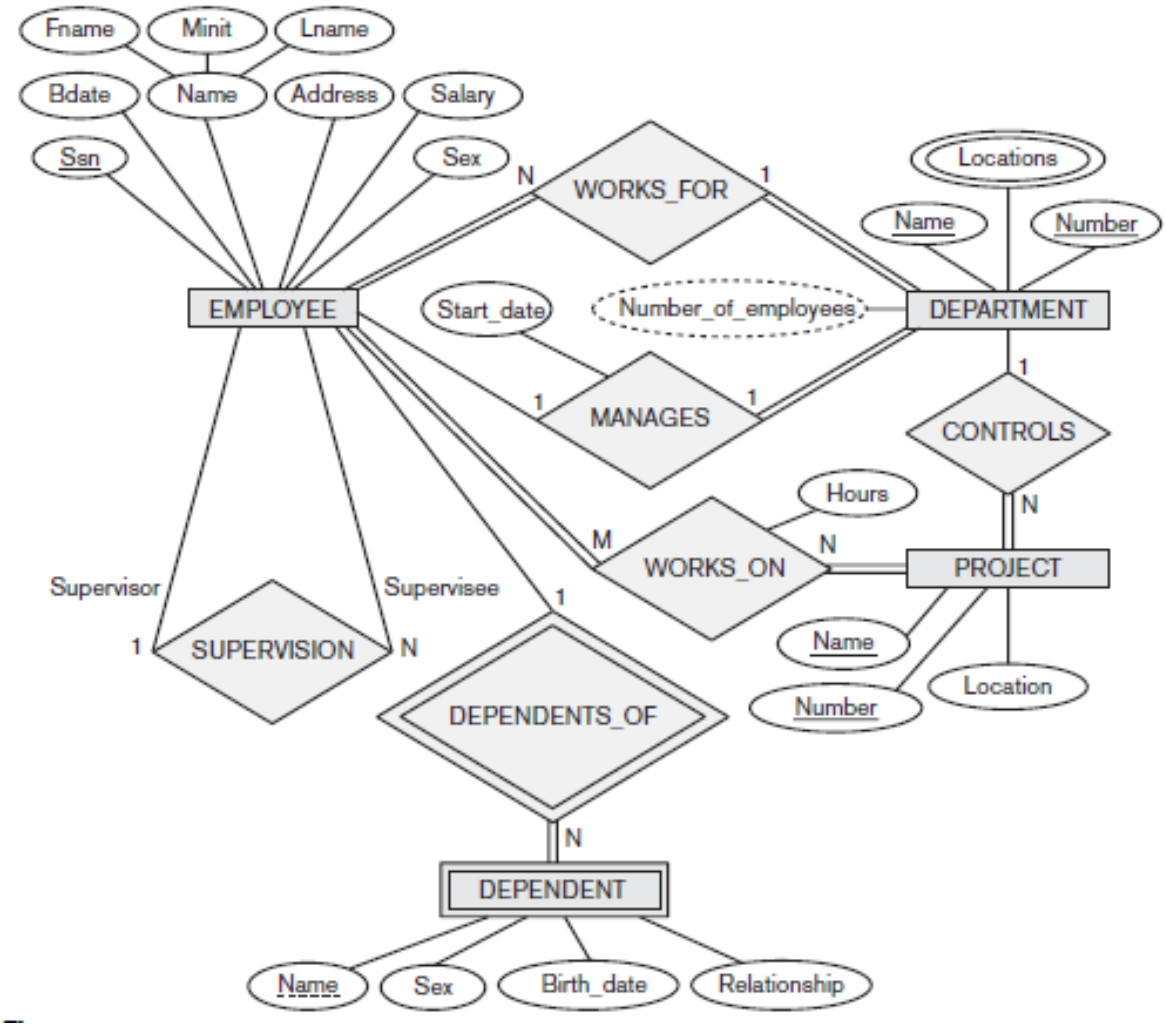


Figure : Entity Relationship (ER) Diagram of COMPANY Database

3 ENTITY TYPES, ENTITY SETS, ATTRIBUTES, AND KEYS

The ER model describes data as entities, relationships, and attributes.

Entities

The basic object that the ER model represents is an **entity,** which is a "thing" in the real world with an independent existence. An entity may be an object with a physical existence—a particular person, car, house, or employee—or it may be an object with a conceptual existence—a company, a job, or a university course.

Attributes : Each entity has **attributes**—the particular properties that describe it. For example, an employee entity may be described by the employee's name, age, address, salary, and job.
A particular entity will have a **value** for each of its attributes. The attribute values that describe each entity become a major part of the data stored in the database.

Ex: 1)The employee entity e1 has four attributes: Name, Address, Age, and HomePhone; their values are "John Smith," "2311 Kirby, Houston, Texas 77001," "55," and "713-749-2630," respectively

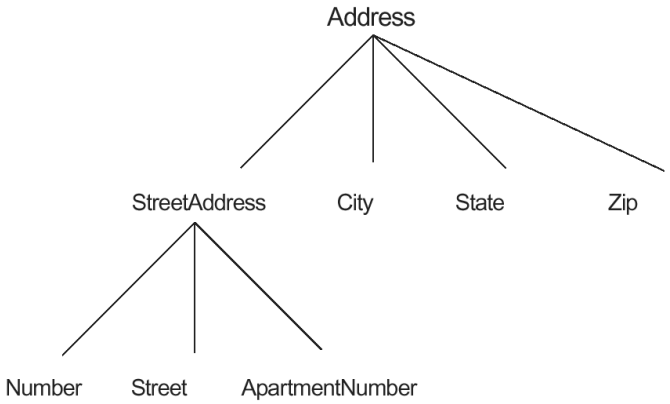
Several types of attributes occur in the ER model:

1) *simple* versus *composite*

2) *single-valued* versus *multi valued*

3) *stored* versus *derived.*

1) Composite attributes can be divided into smaller subparts, which represent more basic attributes with independent meanings.
Ex: Address attribute of the employee entity can be sub-divided into StreetAddress, City, State, and Zip with the values "2311 Kirby," "Houston," "Texas," and "77001."
Composite attributes can form a hierarchy; for example, StreetAddress can be subdivided into three simple attributes, Number, Street, and ApartmentNumber, as shown in Figure .The value of a composite attribute is the concatenation of the values of its constituent simple attributes.



2) Simple or **atomic attributes :** Attributes that are not divisible are called **simple** or **atomic attributes.**

EX: Number, street are simple attributes.

3)Single-valued Attribute : Most attributes have a single value for a particular entity; such attributes are called **single-valued.**
EX: Age is a single-valued attribute of person.

4) Multivalued Attributes : In some cases an attribute can have a set of values for the same entity—for example, a Colors attribute for a car, or a College Degrees attribute for a person. Cars with one color have a single value, whereas two-tone cars have two values for Colors. A multivalued attribute may have lower and upper bounds on the number of values allowed for each individual entity.
Ex: The Colors attribute of a car may have between one and three values, if we assume that a car can have at most three colors.

5)Stored Versus Derived Attributes
In some cases two (or more) attribute values are related—for example, the Age and BirthDate attributes of a person. For a particular person entity, the value of Age can be determined from the current (today's) date and the value of that person's BirthDate. The Age attribute is hence called a **derived attribute** and is said to be **derivable from** the BirthDate attribute, which is called a **stored attribute.**

6)Null Values : In some cases a particular entity may not have an applicable value for an attribute. For example, the ApartmentNumber attribute of an address applies only to addresses that are in apartment buildings and not to other types of residences, such as single-family homes. For such situations, a special value called **null** is created. An address of a single-family home would have null for its ApartmentNumber attribute. Null can also be used if we do not know the value of an attribute for a particular entity—for example, if we do not know the home phone of "John Smith" .The meaning of the former type of null is *not applicable*, whereas the meaning of the latter is *unknown.*
The unknown category of null can be further classified into two cases. The first case arises when it is known that the attribute value exists but is *missing*—for example, if the Height attribute of a person is listed as null.
The second case arises when it is *not known* whether the attribute value exists—for example, if the HomePhone attribute of a person is null.

7)Complex Attributes
Notice that composite and multivalued attributes can be nested in an arbitrary way. We can represent arbitrary nesting by grouping components of a composite attribute between parentheses ( ) and separating the components with commas, and by displaying multivalued attributes between braces {}. Such attributes are called **complex attributes.**

1)For example, PreviousDegrees of a STUDENT is acomposite multi-valued attribute denoted by {PreviousDegrees (College, Year, Degree, Field)}

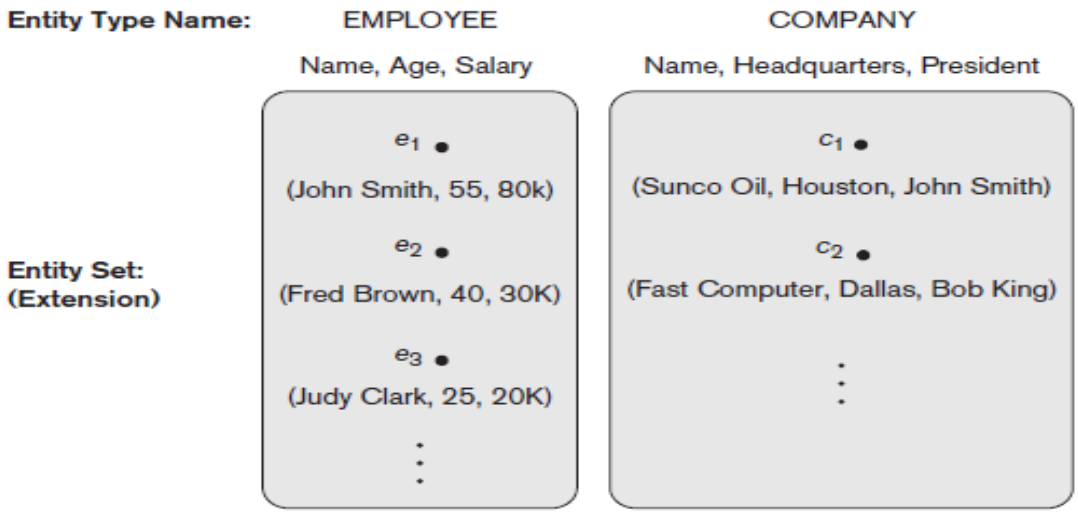
2)If a person can have more than one residence and each residence can have multiple phones, an attribute AddressPhone for a PERSON entity type can be specified as complex attribute.

Entity Types

A database usually contains groups of entities that are similar. For example, a company employing hundreds of employees may want to store similar information concerning each of the employees. These employee entities share the same attributes, but each entity has *its own value(s)* for each attribute.

An **entity type** defines a *collection* (or *set*) of entities that have the same attributes. Each entity type in the database is described by its name and attributes.

Ex:Two entity types, named EMPLOYEE and COMPANY, and a list of attributes for each.



Entity Sets

The collection of all entities of a particular entity type in the database at any point in time is called an **entity set;** the entity set is usually referred to using the same name as the entity type. For example, EMPLOYEE refers to both a *type of entity* as well as the current *set of all employee entities* in the database.

An entity type describes the **schema** or **intension** for a *set of entities* that share the same structure. The collection of entities of a particular entity type are grouped into an entity set, which is also called the **extension** of the entity type.

- An **entity type** is represented in ER diagrams as a **rectangular box** enclosing the entity type name.
- **Attribute** names are enclosed in **ovals** and are attached to their entity type by straight lines.
- **Composite attributes** are attached to their component attributes by straight lines.
- **Multivalued attributes** are displayed in double ovals.
- Each key attribute has its name **underlined** inside the oval

Key Attributes of an Entity Type

An important constraint on the entities of an entity type is the **key** or **uniqueness constraint** on attributes. An entity type usually has an attribute whose values are distinct for each individual entity in the collection. Such an attribute is called a **key attribute,** and its values can be used to identify each entity uniquely.

Ex: 1)The Name attribute is a key of the COMPANY entity type , because no two companies are allowed to have the same name.

2)For the PERSON entity type, a typical key attribute is SocialSecurityNumber.

Sometimes, several attributes together form a key, meaning that the *combination* of the attribute values must be distinct for each entity. If a set of attributes possesses this property, we can define a ***composite attribute*** that becomes a key attribute of the entity type.

Value Sets (Domains) of Attributes

Each simple attribute of an entity type is associated with a **value set** (or **domain** of values), which specifies the set of values that may be assigned to that attribute for each individual entity.

Ex:The range of ages allowed for employees is between 16 and 70, we can specify the value set of the Age attribute of EMPLOYEE to be the set of integer numbers between 16 and 70.

Similarly, we can specify the value set for the Name attribute as being the set of strings of alphabetic characters separated by blank characters and so on. Value sets are not displayed in ER diagrams.

(a)

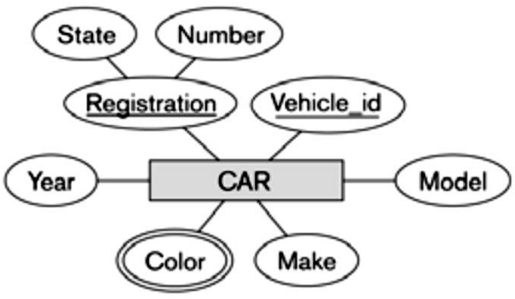


Figure 3.7
The CAR entity type with two key attributes, Registration and Vehicle_id. (a) ER diagram notation. (b) Entity set with three entities.

(b)

CAR
Registration (Number, State), Vehicle_id, Make, Model, Year, {Color}

CAR_1
((ABC 123, TEXAS), TK629, Ford Mustang, convertible, 2004 {red, black})

CAR_2
((ABC 123, NEW YORK), WP9872, Nissan Maxima, 4-door, 2005, {blue})

CAR_3
((VSY 720, TEXAS), TD729, Chrysler LeBaron, 4-door, 2002, {white, blue})

⋮

4 Relationships, Relationship Types, Roles, and Structural Constraints

There are several *implicit relationships* among the various entity types. In fact, whenever an attribute of one entity type refers to another entity type, some relationship exists.
For example, the attribute Manager of DEPARTMENT refers to an employee who manages the department. In the ER model, these references should not be represented as attributes but as **relationships.**

4.1 Relationship Types, Sets and Instances

A **relationship type** *R* among *n* entity types, , . . ., defines a set of associations—or a **relationship set**—among entities from these types. As for entity types and entity sets, a relationship type and its corresponding relationship set are customarily referred to by the *same name R.* Mathematically, the relationship set *R* is a set of **relationship instances.**

Informally, each relationship instance in *R* is an association of entities, where the association includes exactly one entity from each participating entity type. Each such relationship instance represents the fact that the entities participating in are related in some way in the corresponding miniworld situation.

For example, consider a relationship type WORKS_FOR between the two entity types EMPLOYEE and DEPARTMENT, which associates each employee with the department the employee works for. Each relationship instance in the relationship set WORKS_FOR associates one employee entity and one department entity.

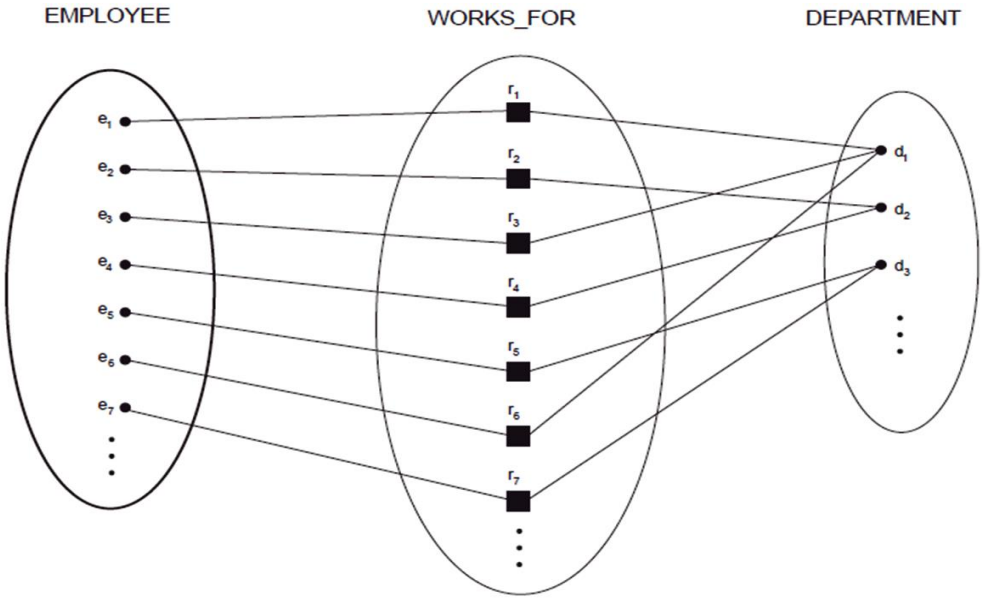


Figure : Some instances in the WORKS_FOR relationship set, which represents a relationship type WORKS_FOR between EMPLOYEE and DEPARTMENT.

The above figure illustrates this example, where each relationship instance is shown connected to the employee and department entities that participate in .*E*mployees e1, e3, and e6 work for department d1; e2 and e4 work for d2; and e5 and e7 work for d3.

In ER diagrams, **relationship types** are displayed as **diamond-shaped boxes**, which are connected by straight lines to the rectangular boxes representing the participating entity types. The relationship name is displayed in the diamond-shaped box

4.2 Degree of a Relationship Type

The **degree** of a relationship type is the number of participating entity types. Hence, the WORKS_FOR relationship is of degree two.

A relationship type of degree one is called **unary**, degree two is called **binary,** and one of degree three is called **ternary.**

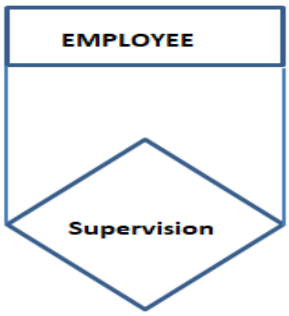


Figure : Unary

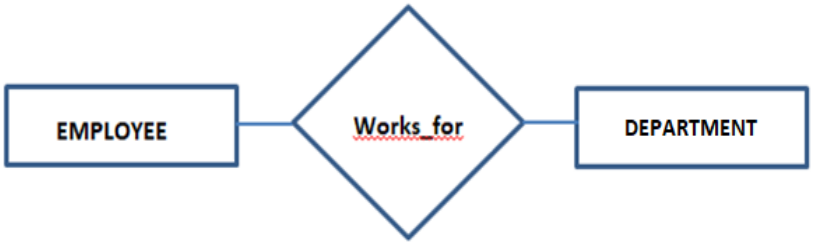


Figure : Binary

Figure : Ternary

An example of a ternary relationship is SUPPLY, shown in above Figure . where each relationship instance associates three entities—a supplier *s*, a part *p*, and a project *j*—whenever *s* supplies part *p* to project *j*. Relationships can generally be of any degree, but the ones most common are binary relationships. Higher-degree relationships are generally more complex than binary relationships.

Relationships as Attributes

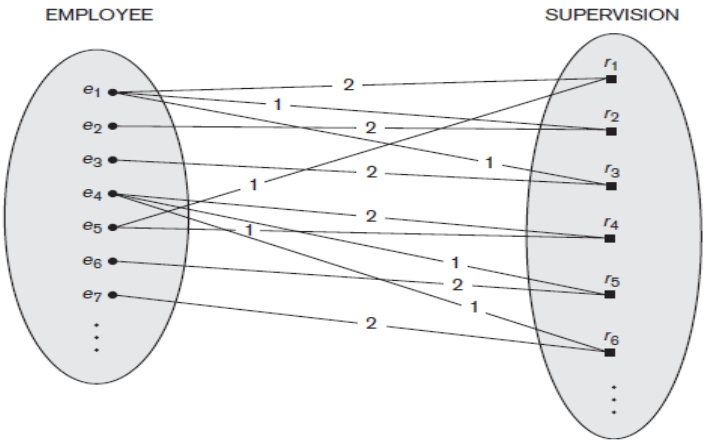
It is sometimes convenient to think of a relationship type in terms of attributes. One can think of an attribute called Department of the EMPLOYEE entity type whose value for each employee entity is (a reference to) the *department entity* that the employee works for.

Role Names and Recursive Relationships

Each entity type that participates in a relationship type plays a particular **role** in the relationship. The **role name** signifies the role that a participating entity from the entity type plays in each relationship instance, and helps to explain what the relationship means.

For example, in the WORKS_FOR relationship type, EMPLOYEE plays the role of *employee* or *worker* and DEPARTMENT plays the role of *department* or *employer*.

Role names are not technically necessary in relationship types where all the participating entity types are distinct, since each entity type name can be used as the role name. However, in some cases the *same* entity type participates more than once in a relationship type in *different roles.* In such cases the role name becomes essential for distinguishing the meaning of each participation. Such relationship types are called **recursive relationships.**



Each relationship instance in SUPERVISION associates two employee entities ej and ek, one of which plays the role of supervisor and the other the role of supervisee.
In the above figure the lines marked "1" represent the supervisor role, and those marked "2" represent the supervisee role; hence, e1 supervises e2 and e3; e4 supervises e6 and e7; and e5 supervises e1 and e4.

figure shows an example. The SUPERVISION relationship type relates an employee to a supervisor, where both employee and supervisor entities are members of the same EMPLOYEE entity type. Hence, the EMPLOYEE entity type *participates twice* in SUPERVISION: once in the role of *supervisor* (or boss), and once in the role of *supervisee* (or subordinate).

4.3 CONSTRAINTS ON RELATIONSHIP TYPES

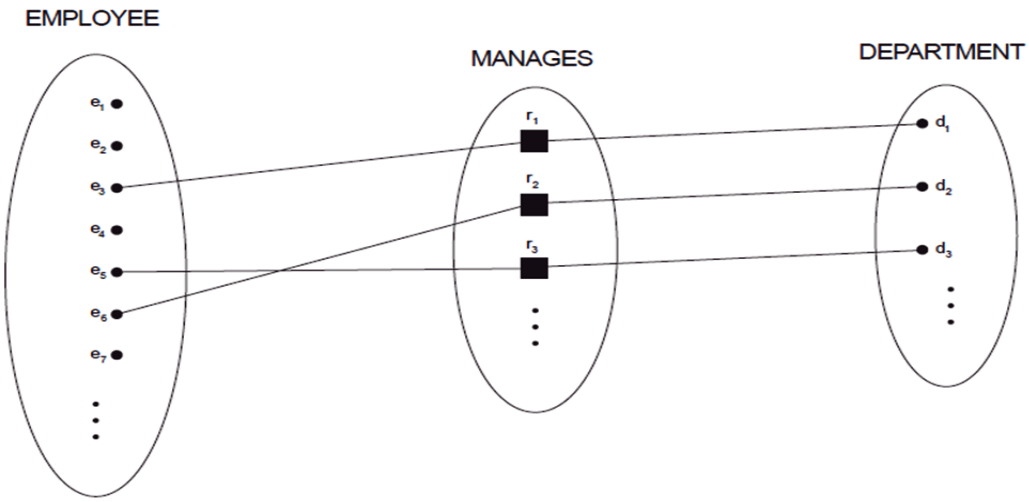
Cardinality Ratios for Binary Relationships Participation Constraints and Existence Dependencies. Relationship types usually have certain constraints that limit the possible combinations of entities that may participate in the corresponding relationship set. These constraints are determined from the mini world situation that the relationships represent.
For example, if the company has a rule that each employee must work for exactly one department, then we would like to describe this constraint in the schema. We can distinguish two main types of relationship constraints: ***cardinality ratio*** and ***participation.***

Cardinality Ratios for Binary Relationships

Cardinality ratio : Cardinality ratio for a binary relationship specifies the number of relationship instances that an entity can participate in. The possible cardinality ratios for binary relationship types are 1:1, 1:N, N:1, and M:N. Cardinality ratios for binary relationships are displayed on ER diagrams by displaying 1, M, and N on the diamonds.

1:1 Binary Relationship

An example of a 1:1 binary relationship is MANAGES which relates a department entity to the employee who manages that department. This represents the miniworld constraints that an employee can manage only one department and that a department has only one manager.

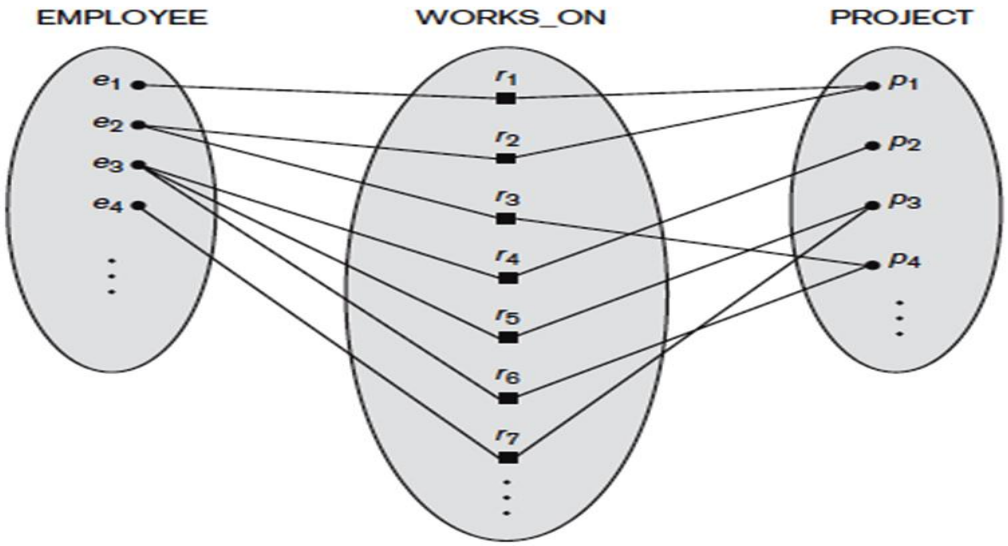


1: N Binary Relationship

For example, in the WORKS_FOR binary relationship type, DEPARTMENT:EMPLOYEE is of cardinality ratio 1:N, meaning that each department can be related to numerous employees but an employee can be related to only one department.

M: N Relationship

The relationship type WORKS_ON is of cardinality ratio M:N, because the miniworld rule is that an employee can work on several projects and a project can have several employees.



Participation Constraints and Existence Dependencies

The **participation constraint** specifies whether the existence of an entity depends on its being related to another entity via the relationship type. There are two types of participation constraints—**total** and **partial**—which we illustrate by example.

a) If a company policy states that *every* employee must work for a department, then an employee entity can exist only if it participates in a WORKS_FOR relationship instance .

Thus, the participation of EMPLOYEE in WORKS_FOR is called **total participation,** meaning that every entity in "the total set" of employee entities must be related to a department entity via WORKS_FOR. Total participation is also called **existence dependency.**

b) we do not expect every employee to manage a department, so the participation of EMPLOYEE in the MANAGES relationship type is **partial,** meaning that *some* or "part of the set of" employee entities are related to a department entity via MANAGES, but not necessarily all.

Structural Constraints

cardinality ratio and participation constraints, taken together, as the **structural constraints** of a relationship type.

In ER diagrams, total participation is displayed as a *double line* connecting the participating entity type to the relationship, whereas partial participation is represented by a *single line* .

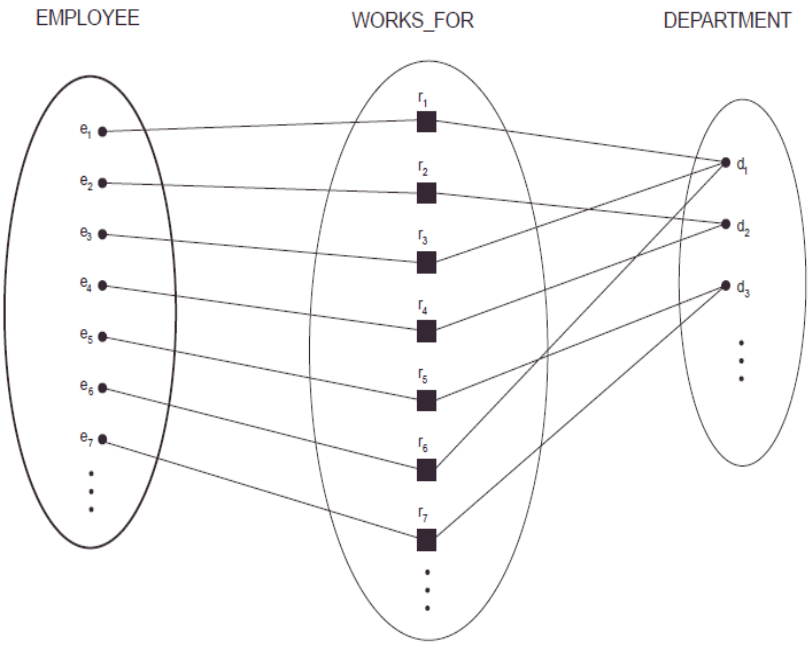


Figure : Total Participation

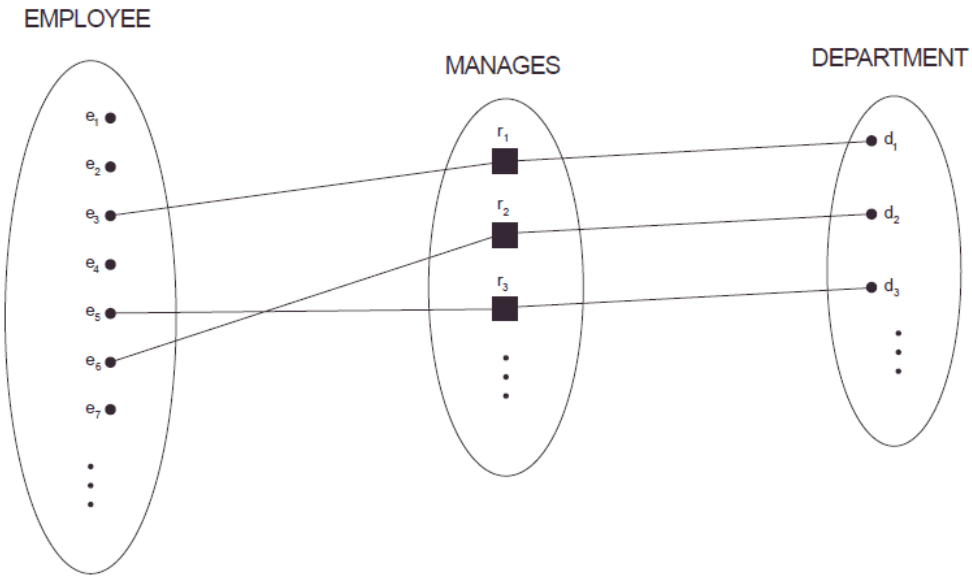


Figure: Partial Participation

Attributes of Relationship Types

Relationship types can also have attributes, similar to those of entity types. For example, to record the number of hours per week that an employee works on a particular project, we can include an attribute **Hours** for the WORKS_ON relationship type .

Another example is to include the date on which a manager started managing a department via an attribute **StartDate**for the MANAGES relationship type .

Notice that attributes of 1:1 or 1:N relationship types can be migrated to one of the participating entity types. For example, the StartDate attribute for the MANAGES relationship can be an attribute of either EMPLOYEE or DEPARTMENT—although conceptually it belongs to

MANAGES. This is because MANAGES is a 1:1 relationship, so every department or employee entity participates in *at most one* relationship instance. Hence, the value of the StartDate attribute can be determined separately, either by the participating department entity or by the participating employee (manager) entity.

For a 1:N relationship type, a relationship attribute can be migrated *only* to the entity type at the N-side of the relationship. For example if the WORKS_FOR relationship also has an attribute StartDate that indicates when an employee started working for a department, this attribute can be included as an attribute of EMPLOYEE. This is because each employee entity participates in at most one relationship instance in WORKS_FOR. In both 1:1 and 1:N relationship types, the decision as to where a relationship attribute should be placed—as a relationship type attribute or as an attribute of a participating entity type—is determined subjectively by the schema designer.

For M:N relationship types, some attributes may be determined by the *combination of participating entities* in a relationship instance, not by any single entity. Such attributes *must be specified as relationship attributes.* An example is the Hours attribute of the M:N relationship WORKS_ON . the number of hours an employee works on a project is determined by an employee-project combination and not separately by either entity.

5 Weak Entity Types

Entity types that do not have key attributes of their own are called **weak entity types.** In contrast, **regular entity types** that do have a key attribute are sometimes called **strong entity types.**

Entities belonging to a weak entity type are identified by being related to specific entities from another entity type in combination with some of their attribute values. We call this other entity type the **identifying** or **owner entity type** and we call the relationship type that relates a weak entity type to its owner the **identifying relationship** of the weak entity type .

A weak entity type always has a *total participation constraint* (existence dependency) with respect to its identifying relationship, because a weak entity cannot be identified without an owner entity.

Consider the entity type DEPENDENT, related to EMPLOYEE, which is used to keep track of the dependents of each employee via a 1:N relationship .The attributes of DEPENDENT are Name (the first name of the dependent), BirthDate, Sex, and Relationship (to the employee). Two dependents of *two distinct employees* may, by chance, have the same values for Name, BirthDate, Sex, and Relationship, but they are still distinct entities. They are identified as distinct entities only after determining the *particular employee entity* to which each dependent is related. Each employee entity is said to **own** the dependent entities that are related to it.

A weak entity type normally has a **partial key,** which is the set of attributes that can uniquely identify weak entities that are *related to the same owner entity* .In our example, if we assume

that no two dependents of the same employee ever have the same first name, the attribute Name of DEPENDENT is the partial key. In the worst case, a composite attribute of *all the weak entity's attributes* will be the partial key.

In ER diagrams, both a weak entity type and its identifying relationship are distinguished by surrounding their **boxes and diamonds with *double lines***. The partial key attribute is underlined with a dashed or dotted line.

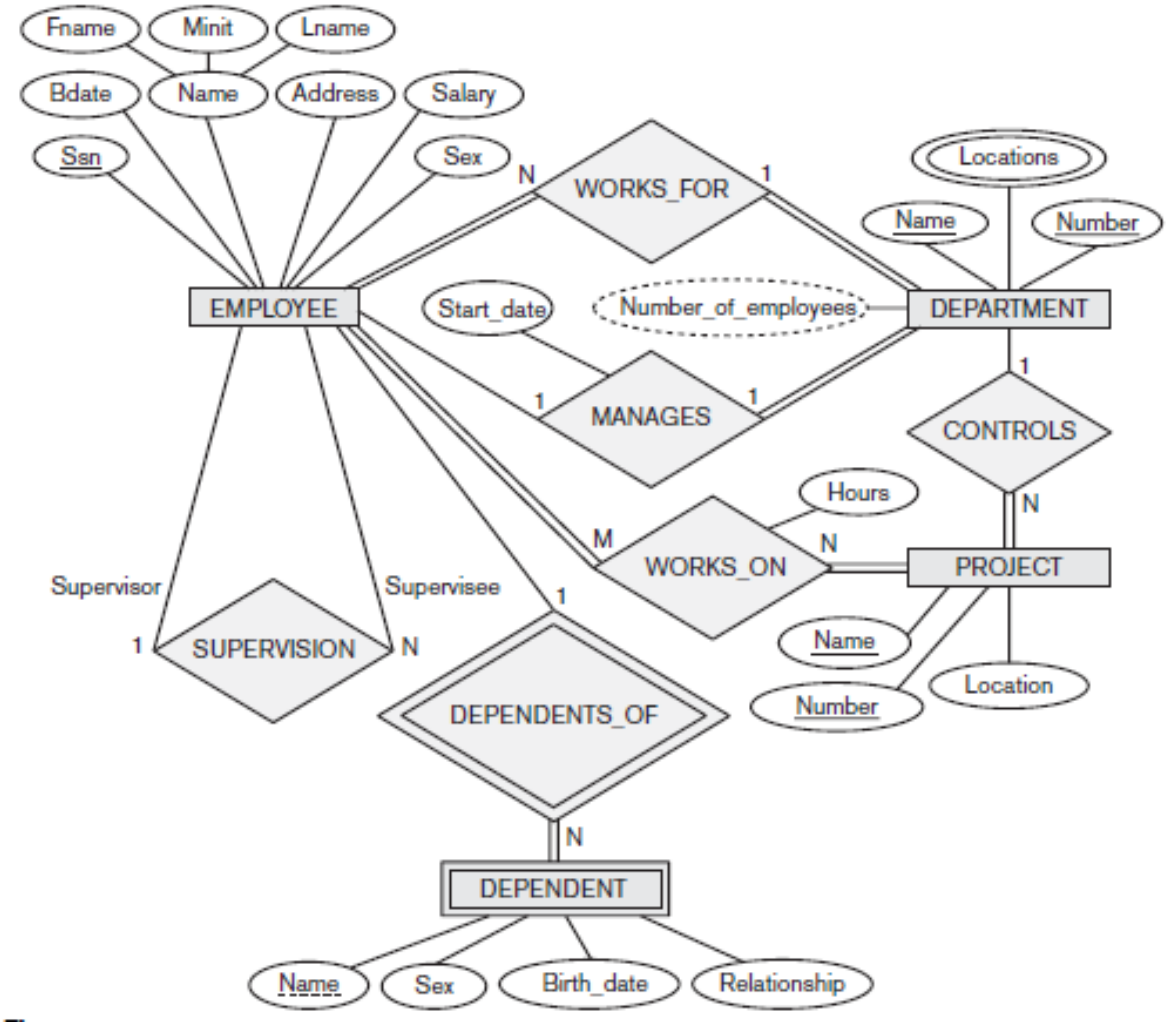
6 ER Design for the COMPANY Database

- Entity types such as EMPLOYEE, DEPARTMENT, and PROJECT are shown in rectangular boxes.
- Relationship types such as WORKS_FOR, MANAGES, CONTROLS, and WORKS_ON are shown in diamond-shaped boxes attached to the participating entity types with straight lines.
- Attributes are shown in ovals, and each attribute is attached by a straight line to its entity type or relationship type.
- Component attributes of a composite attribute are attached to the oval representing the composite attribute, as illustrated by the Name attribute of EMPLOYEE.
- Multivalued attributes are shown in double ovals, as illustrated by the Locations attribute of DEPARTMENT.
- Key attributes have their names underlined.
- Derived attributes are shown in dotted ovals, as illustrated by the NumberOfEmployees attribute of DEPARTMENT.
- Weak entity types are distinguished by being placed in double rectangles and by having their identifying relationship placed in double diamonds, as illustrated by the DEPENDENT entity type and the DEPENDENTS_OF identifying relationship type.
- The partial key of the weak entity type is underlined with a dotted line.

- In our example, we specify the following relationship types:

1. MANAGES, a 1:1 relationship type between EMPLOYEE and DEPARTMENT. EMPLOYEE participation is partial. DEPARTMENT participation is not clear from the requirements. We question the users, who say that a department must have a manager at all times, which implies total participation .The attribute StartDate is assigned to this relationship type.
2. WORKS_FOR, a 1:N relationship type between DEPARTMENT and EMPLOYEE. Both participations are total.
3. CONTROLS, a 1:N relationship type between DEPARTMENT and PROJECT. The participation of PROJECT is total, whereas that of DEPARTMENT is determined to be partial, after consultation with the users.

4. SUPERVISION, a 1:N relationship type between EMPLOYEE (in the supervisor role) and EMPLOYEE (in the supervisee role). Both participations are determined to be partial, after the users indicate that not every employee is a supervisor and not every employee has a supervisor.
5. WORKS_ON, determined to be an M:N relationship type with attribute Hours, after the users indicate that a project can have several employees working on it. Both participations are determined to be total.
6. DEPENDENTS_OF, a 1:N relationship type between EMPLOYEE and DEPENDENT, which is also the identifying relationship for the weak entity type DEPENDENT. The participation of EMPLOYEE is partial, whereas that of DEPENDENT is total.



In Figure the cardinality ratio of each *binary* relationship type is specified by attaching a 1, M, or N on each participating edge. The cardinality ratio of DEPARTMENT: EMPLOYEE in MANAGES is 1:1, whereas it is 1:N for DEPARTMENT:EMPLOYEE in WORKS_FOR, and it is M:N for WORKS_ON. The participation constraint is specified by a single line for partial participation and by double lines for total participation (existence dependency).

In Figure we show the role names for the SUPERVISION relationship type because the EMPLOYEE entity type plays both roles in that relationship. Notice that the cardinality is 1:N

from supervisor to supervisee because, on the one hand, each employee in the role of supervisee has at most one direct supervisor, whereas an employee in the role of supervisor can supervise zero or more employees.